Banco de dados

SQLite, DBeaver, ORMs e Prisma

Sumário

BANCO DE DADOS

Conceito de banco de dados relacional

Introdução ao SQLite

Conhecendo ferramentas SGBD

DBeaver

ORM

Conceito · Usabilidade · Cuidados

PRISMA

Conhecendo o Prisma ORM

O que mudou no Prisma 7

Instalando o Prisma ORM no nosso projeto

Configurando conexão com banco de dados através do Prisma

Migrations e Prisma Client

Cuidados ao desenvolver (npx prisma migrate dev)

CRUD com o Prisma Client

create · find · update · delete

Relembrando

Banco de dados relacional

Estruturado em tabelas

Usam chaves primárias e estrangeiras para relacionar tabelas

Utilizam SQL

Possuem schema fixo (campos definidos)

`MySQL, Postgres, SQLite, MariaDB, Oracle, SQL Server…`

Banco de dados não relacional

Podem seguir esquema de arquivos, colunar, chave-valor, grafos...

Permitem schemas dinâmicos (campos variam)

Otimizado para grandes volumes de leitura e escrita

`Cassandra, DynamoDB, MongoDB, Redis, Neo4j`

SQLite

Banco de dados baseado em arquivo

Fácil manipulação

Fácil sintaxe (SQL)

OpenSource (código aberto)

```
CRUD-DB
  generated/prisma
  node_modules
  prisma
    migrations
    schema.prisma
  .env
  dev.db
  package-lock.json
  package.json
  prisma7.config.ts
  server.js
```

O banco inteiro é um arquivo dentro do projeto: **dev.db**

Instalando o SQLite

sqlite.org

LATEST RELEASE

Version 3.50.4 (2025-07-30)

Download | Prior Releases

Usando o WINGET

INSTALANDO

1 Botão Windows

2 Powershell

3 Digitar:

```
winget install SQLite.SQLite
```

VERIFICANDO A VERSÃO

1 Feche o terminal e abra um novo

2 Digite o comando:

```
sqlite3 --version
```

Caso falhe o WINGET: o manual de instalação de forma manual está no GitHub, nos arquivos auxiliares.

SGBD

Sistema Gerenciador de Banco de Dados

Intermediário entre o banco de dados e o usuário que o utiliza.

Gerencia a criação, acesso, manipulação, segurança e organização dos dados.

> *Interface bonitinha pra você usar o banco de dados*

Instalando o DBeaver

https://dbeaver.io/download/

DOWNLOAD

DBeaver Community 25.2.0

Lançado em 31 de agosto de 2025. Gratuito e open source.

Windows: instalador (installer) ou arquivo zip.

Importante!

Na instalação do DBeaver, na tela **Escolher componentes**:

`DBeaver Community`	Marcado (obrigatório)
`Include Java`	Marcado
`Reset Settings`	Desmarcado
`Associate SQL files`	Desmarcado
`Associate SQLite database files`	**Marcado (não esqueça)**

Conectando ao banco com o DBeaver

01 Nova conexão

02 Escolher o tipo de banco (SQLite)

03 Selecionar um arquivo de banco de dados (vamos criar um como exemplo)

CASO SE DEPAREM COM A JANELA "CONFIGURAÇÕES DE DRIVER"

Basta clicar em **"Download"** que os pacotes serão baixados.

02

ORMs

O que é um ORM

Object-Relational Mapping (Mapeamento objeto-relacional)

É uma espécie de "ponte" entre a programação em si e o banco de dados.



Permite que você utilize funções pré-escritas ao invés de escrever um SQL puro.

Para que serve

01 Abstração de SQL: você escreve código, programa, ao invés de SQL

02 Possuem proteção automática contra *sql injection*

03 Desenvolvimento mais rápido

04 Facilita a mudança entre diferentes bancos de dados

Cuidados e desvantagens

Performance inferior

Pode gerar queries demoradas

Queries complexas

Algumas consultas avançadas (muitos joins) podem acabar se tornando mais complexas usando ORM do que usando SQL

Curva de aprendizagem

Cada ORM tem sua própria linguagem e suas próprias funções

Dependência

Seu código fica acoplado ao ORM

Como eu sei o recurso que cada um tem? Preciso decorar tudo?

Não precisa decorar.

01

Documentação oficial

02

Bookmarks

03

Praticar principais métodos (CRUD)

03

Prisma ORM

Estrutura básica

```
CRUD-DB
  generated/prisma
  node_modules
  prisma
    migrations
    schema.prisma
  .env
  dev.db
  package-lock.json
  package.json
  prisma7.config.ts
  server.js
```

schema.prisma

O arquivo principal do Prisma, dentro da pasta **prisma**, junto com as **migrations**.

O **dev.db** e o **prisma7.config.ts** ficam na raiz do projeto. A pasta **generated/prisma** guarda o client gerado.

schema.prisma

CLIENT E BANCO

```
generator client {
  provider = "prisma-client"
  output   = "../generated/prisma"
}



datasource db {
  provider = "sqlite"
}
```

A URL do banco não fica mais aqui. Ela está no **prisma7.config.ts**.

TABELA

```
model User {
  id    Int     @id @default(autoincrement())
  nome  String
  email String  @unique


  @@map("usuarios")
}
```

Exemplo completo: schema de livros

prisma/schema.prisma

```
generator client {
  provider = "prisma-client"
  output   = "../generated/prisma"
}



datasource db {
  provider = "sqlite"
}



model Livro {
  id            Int     @id @default(autoincrement())
  titulo        String
  autor         String
  fgdisponivel  Boolean @default(true)


  @@map("livros")
```

`id`

Número inteiro, chave primária, gerado automaticamente (1, 2, 3…)

`titulo, autor`

Textos obrigatórios (**String**)

`fgdisponivel`

Flag verdadeiro/falso. Começa como **true**. No SQLite vira 0 ou 1; o Prisma converte para você

`@@map("livros")`

Nome da tabela no banco. Sem isso, seria **Livro**

Conhecendo o Prisma (documentação)

Prisma Documentation

A documentação abre na versão mais nova do Prisma. Nesta aula usamos a **versão 7**, então procure o caminho com **v7**.

Quickstart com SQLite:

```
prisma.io/docs/v7/prisma-orm/quickstart/sqlite
```

Vantagens do Prisma

01

Type-safe

Gera um *client* específico, do tipo do schema

02

Queries otimizadas

03

Schema declarativo

04

Boa experiência de desenvolvimento

Sessões importantes

1	Getting started	Tutorial passo a passo
2	Schema reference	Referência da sintaxe
3	Client API	Como usar o Prisma Client
4	Migrate	Sistema de migrações
5	Examples	Projetos prontos para estudar

`schema.prisma`

Arquivo principal do Prisma

DEFINE

- Tabelas
- Relacionamentos
- Índices
- Qual banco usar (provider)
- Onde gerar o client (generator)

> A **URL de conexão** não fica no schema: ela está no prisma7.config.ts.

O que mudou no Prisma 7

Arquivo de configuração

Antes só o schema.prisma

Agora mais o prisma7.config.ts

URL do banco

Antes dentro do schema.prisma

Agora no config, lida do DATABASE_URL

Arquivo .env

Antes carregado automaticamente

Agora você carrega com o dotenv

Generator

Antes prisma-client-js

Agora prisma-client, com output

Criar o client

Antes new PrismaClient()

Agora new PrismaClient({ adapter })

Gerar o client

Antes o migrate dev já gerava

Agora você roda o prisma generate

Instalando o Prisma

```bash
# CLI do Prisma (init, migrate, generate)

npm install prisma@7 --save-dev



# Client, adapter do SQLite e leitor do .env

npm install @prisma/client@7 @prisma/adapter-better-sqlite3 dotenv
```

SEMPRE COM @7

Sem o número, o npm pode instalar a versão 8, ainda em testes, que funciona de outro jeito.

NO PACKAGE.JSON

Inclua "type": "module". O Prisma 7 usa módulos ES (import).

Inicializando o Prisma no projeto

BASH

```
npx prisma init --datasource-provider sqlite --output ../generated/prisma
```

ESSE COMANDO CRIA

`prisma/`
A pasta com o schema.prisma

`prisma7.config.ts`
A configuração do CLI

`.env`
Com a DATABASE_URL do dev.db

`.gitignore`
Ignora o .env e o client gerado

> O init **não cria o banco**. O arquivo dev.db só aparece no primeiro migrate.

Configurando a conexão com o banco

PRISMA7.CONFIG.TS

```ts
import "dotenv/config";

import { defineConfig } from "prisma/config";



export default defineConfig({
  schema: "prisma/schema.prisma",
  migrations: {
    path: "prisma/migrations",
  },
  datasource: {
    url: process.env["DATABASE_URL"],
  },
});
```

.ENV

```
DATABASE_URL="file:./dev.db"
```

`import "dotenv/config"`
Carrega o .env. O Prisma 7 não faz isso sozinho.

`datasource.url`
De onde vem a URL do banco.

`migrations.path`
Onde ficam as migrations geradas.

04

Migrations

O que são migrations

Scripts de **versionamento de banco de dados**, que permitem:

01

Evoluir o schema (criar novos campos, novas tabelas, etc.) de forma controlada

02

Sincronizar mudanças entre ambientes

03

Manter histórico de alterações

04

Reverter mudanças quando necessário

Como aplicar mudanças em tabelas

O Prisma compara o **schema.prisma** com o estado do seu banco atual.

Quando ele encontra algo que mudou, realiza o seguinte processo:

01

Gera o SQL necessário para atualizar o seu banco

02

Cria um arquivo de migration automaticamente

03

Aplica as mudanças

> Logo: para atualizar um banco, basta atualizar o arquivo *schema.prisma*.
>
> Depois, rode npx prisma generate para atualizar também o client.

Executando migrations

BASH

```
npx prisma migrate dev --name init
```

ESSE COMANDO

01

Cria o dev.db, se ele ainda não existir

02

Grava o SQL em prisma/migrations e cria as tabelas

03

Não gera mais o client. Isso é o próximo passo

Gerando o Prisma Client

BASH

```
npx prisma generate
```

```
Generated Prisma Client (7.10.1) to ./generated/prisma
```

O que ele faz

Lê o schema.prisma e cria o client, com as funções do seu modelo, na pasta generated/prisma.

Quando rodar

Depois do primeiro migrate e a cada mudança no schema.

Criando o client com o adapter

PRISMA.JS, NA RAIZ DO PROJETO

```
import "dotenv/config";

import { PrismaBetterSqlite3 } from
  "@prisma/adapter-better-sqlite3";

import { PrismaClient } from
  "./generated/prisma/client.ts";



const adapter = new PrismaBetterSqlite3({
  url: process.env.DATABASE_URL,
});



export const prisma = new PrismaClient({ adapter });
```

USANDO EM QUALQUER ARQUIVO

```
import { prisma } from
  "./prisma.js";



const ana = await prisma.user
  .create({ data: {
    nome: "Ana",
    email: "ana@email.com",
  }});
```

> O client gerado é TypeScript. No **Node 22.18 ou superior** o JavaScript importa o .ts direto. Em versões mais antigas, rode com npx tsx.

Atenção

UPGRADE

O Prisma realiza o upgrade do banco de dados automaticamente, através dos migrations.

DOWNGRADE

Mas não possui downgrade: é preciso realizar essa ação manualmente, seja com um SQL “manual” ou algum outro procedimento.

PRÁTICA

Vamos tentar na prática

`npx prisma init --datasource-provider sqlite --output ../generated/prisma`

`schema.prisma`

`npx prisma migrate dev --name init`

`npx prisma generate`

05

CRUD

Create, Read, Update, Delete com o Prisma Client

As funções do Prisma Client

SINTAXE

```
await prisma.livro.função({ opções })
```

C

Create

`create`
`createMany`

R

Read

`findMany`
`findUnique`
`findFirst`

U

Update

`update`
`updateMany`

D

Delete

`delete`
`deleteMany`

> **prisma.livro**: o nome do model com a primeira letra minúscula. Toda função devolve uma **Promise**, então use **await**.

Como chamar: prisma.nomeDoModel

NO SCHEMA.PRISMA

```
model Livro {
  id     Int @id
  titulo String
  ...
}
```

NO CÓDIGO

```
await prisma.livro.findMany()
```

model Livro	model User	model ItemPedido
`prisma.livro`	`prisma.user`	`prisma.itemPedido`

> Use o **nome do model** com a **primeira letra minúscula**.
>
> Não é o nome da tabela: mesmo com @@map("livros"), o código continua prisma.livro.

Create: criar registros

`create`

Cria **um** registro. O valor vai em **data**.

```
const livro = await prisma.livro
  .create({
    data: {
      titulo: "Dom Casmurro",
      autor: "Machado de Assis",
    },
  });
```

DEVOLVE O LIVRO CRIADO

```
{ id: 1, titulo: 'Dom Casmurro', ... }
```

`createMany`

Cria **vários** de uma vez. O **data** recebe uma lista.

```
const r = await prisma.livro
  .createMany({
    data: [
      { titulo: "Vidas Secas", autor: "G. Ramos" },
      { titulo: "O Cortiço", autor: "A. Azevedo" },
    ],
  });
```

DEVOLVE SÓ A QUANTIDADE

```
{ count: 2 }
```

Read: buscar vários

findMany()

```
// todos os livros (um array)

const livros = await
  prisma.livro.findMany();
```

findMany({ where })

```
// só os disponíveis

const livros = await
  prisma.livro.findMany({
    where: { fgdisponivel: true },
  });
```

count()

```
// quantos livros existem: 4

const total = await
  prisma.livro.count();
```

count({ where })

```
// quantos estão disponíveis: 3

const total = await
  prisma.livro.count({
    where: { fgdisponivel: true },
  });
```

> O **findMany** sempre devolve um **array**. Se nada combinar, devolve [] (lista vazia).

Read: buscar um só

`findUnique`

Busca por um campo **único**: o id ou um campo com @unique.

```
const livro = await prisma.livro
  .findUnique({
    where: { id: 1 },
  });
```

SE ENCONTRAR

Devolve **um objeto**: o livro.

`findFirst`

Devolve o **primeiro** que combina com o filtro, em qualquer campo.

```
const livro = await prisma.livro
  .findFirst({
    where: { autor: "Machado de Assis" },
  });
```

SE NÃO ENCONTRAR

Devolve **null**, sem erro.

```
if (!livro) { // confira antes de usar
  return res.status(404).json({ erro: "Livro não encontrado" });
}
```

Filtros com where

`{ autor: "Machado de Assis" }`	igual (valor exato)
`{ titulo: { contains: "Dom" } }`	contém o texto
`{ titulo: { startsWith: "Vidas" } }`	começa com (e endsWith: termina com)
`{ id: { gt: 2 } }`	maior que (lt, gte e lte também)
`{ id: { in: [1, 3] } }`	está dentro da lista
`{ autor: "X", fgdisponivel: true }`	vários campos juntos: **E**
`{ OR: [{ autor: "X" }, { autor: "Y" }] }`	um **OU** o outro
`{ NOT: { autor: "X" } }`	negação: tudo, menos isso

Funciona em **findMany**, **findFirst**, **count**, **updateMany** e **deleteMany**.

Mais opções para a busca

`select: { titulo: true }`

devolve só os campos escolhidos

`orderBy: { titulo: "asc" }`

ordena (asc: A–Z, desc: Z–A)

`take: 10`

limita a quantidade de resultados

`skip: 10`

pula os primeiros (paginação)

COMBINANDO TUDO

```
const livros = await prisma.livro
  .findMany({
    where: { fgdisponivel: true },
    orderBy: { titulo: "asc" },
    take: 10,
  });
```

Os disponíveis, em ordem alfabética, no máximo 10.

A ordem das opções dentro do objeto **não importa**.

Update: atualizar registros

`update`

Atualiza **um** registro, encontrado pelo id ou campo único.

```
const livro = await prisma.livro
  .update({
    where: { id: 3 },
    data: { fgdisponivel: true },
  });
```

DEVOLVE O LIVRO JÁ ATUALIZADO

```
{ id: 3, ..., fgdisponivel: true }
```

`updateMany`

Atualiza **todos** os que combinam com o filtro.

```
const r = await prisma.livro
  .updateMany({
    where: { autor: "Machado de Assis" },
    data: { fgdisponivel: false },
  });
```

DEVOLVE QUANTOS MUDARAM

```
{ count: 2 }
```

`where`

quem vai ser atualizado

`data`

o que vai mudar (só os campos citados)

Delete: apagar registros

`delete`

Apaga **um** registro, pelo id ou campo único.

```
const livro = await prisma.livro
  .delete({
    where: { id: 4 },
  });
```

DEVOLVE O LIVRO APAGADO

```
{ id: 4, titulo: 'O Cortiço', ... }
```

`deleteMany`

Apaga **todos** os que combinam com o filtro.

```
const r = await prisma.livro
  .deleteMany({
    where: { fgdisponivel: false },
  });
```

DEVOLVE QUANTOS FORAM APAGADOS

```
{ count: 2 }
```

Cuidado

Sem o **where**, o **deleteMany()** apaga **todos** os registros da tabela. O **updateMany()** também atinge todos.

Cuidados com o CRUD

CAPTURANDO O ERRO

```
try {
  await prisma.livro.delete({
    where: { id: 99 },
  });
} catch (erro) {
  console.log(erro.code);
  // P2025: registro não existe
}
```

Esquecer o await

A variável guarda uma Promise pendente, não o resultado.

Registro que não existe

update e delete dão erro (P2025). findUnique devolve null.

Valor repetido

Um create com campo @unique repetido dá erro (P2002).

Many sem where

updateMany e deleteMany sem filtro afetam a tabela inteira.

O código do erro fica em erro.code. Com ele dá para responder a cada situação.